University of Zurich[UZH]

ETH
Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Working title

Master Thesis

Anej Rozman

Advisors: Prof. Patrick Cheridito, Urban Ulrych

Department of Mathematics, ETH Zürich

Acknowledgments

Abstract

We present a randomized algorithm for selecting the median of an unsorted array in $\mathcal{O}(n)$ time with high probability. Unlike classical deterministic methods—notably the median-of-medians algorithm—our approach leverages random sampling to simplify the pivot-selection process and reduce overhead. Specifically, we extract a sublinear sample of the input, sort this sample, and employ its near-median elements as pivots. By bounding $\mathbb{P}[|X - \mathbb{E}[X]| \geq k]$ for appropriate choices of X and k, we ensure that the size of the sub-array requiring a full sort remains sufficiently small. A rigorous analysis, based on Chebyshev's inequality, demonstrates that the algorithm's probability of "failure" (i.e., incorrectly bracketed pivots) decays quickly with the input size. In successful executions, the asymptotic cost of sorting the sample and the sub-array is dominated by a single pass through the input, yielding an overall running time of $\mathcal{O}(n)$. While deterministic linear-time solutions to selection exist, our randomized method offers competitive performance in practice, owing to its simplicity and favorable hidden constants. We also discuss how this technique can be combined with deterministic strategies to eliminate worst-case risk.

Contents

Contents **iii**

1 Introduction **1**

2 Literature Review **3**

3 Notation and Preliminaries **4**

4 Algorithm and Analysis **7**

5 Conclusion **12**

6 Model-free multi-currency portfolio returns **14**
6.1 Unhedged Portfolio Returns 14
6.2 Hedged Portfolio Returns 15
6.3 Matrix Representation and Decomposition 15

Bibliography **17**

Appendices **18**

ES Optimization Equivalence **19**

ES Optimization Implementation **21**
Sample-Based Expected Shortfall 21
Linearization of the L_1 Penalty and Leverage 22
Quadratic Translation of the L_2 Penalty 22
Separate Optimization (Currency Overlay) 22
Step 1: Asset-Level Optimization 22
Step 2: Currency-Level Optimization 23
Joint Optimization Framework 23

Chapter 1

Introduction

The median of a dataset is a fundamental measure of central tendency that arises in numerous computational and statistical contexts. In an array of n elements, indexed from 1 to n, the median can be intuitively understood as the "middle" element in the sorted order if n is odd, or the average of the two middle elements if n is even. While this definition naturally suggests sorting the entire array, doing so in $\mathcal{O}(n \log(n))$ time is often unnecessary when the goal is to identify only the median.

Motivation and Previous Work

Several well-known algorithms exist for selecting the median or, more generally, the k-th smallest element of an array. A straightforward method is to sort the array and then pick the median in constant time. However, this approach has a runtime of $\mathcal{O}(n \log(n))$, which can be excessive for large n. More advanced methods, such as the *median-of-medians* algorithm introduced by Blum, Floyd, Pratt, Rivest, and Tarjan [1], achieve a worst-case runtime of $\mathcal{O}(n)$ using a deterministic divide-and-conquer strategy. While optimal in the worst case, the median-of-medians algorithm involves significant overhead, making it less practical in many scenarios. Alternatively, randomized algorithms like Quickselect often achieve linear-time performance on average but can degrade to $\mathcal{O}(n^2)$ in the worst case without additional precautions.

Our Contribution

In this thesis, we present a *randomized* algorithm that computes the median in $\mathcal{O}(n)$ time *with high probability*, thereby combining practical efficiency with strong theoretical guarantees. The key idea is to leverage a carefully chosen random sample of the input array:

1. We extract a subset of elements (the "sample") whose size grows sublinearly with n.
2. Using this sample, we identify pivot elements that, with high probability, closely approximate the true median's position.
3. We then reduce the problem to sorting only a small subset of the original array, thereby achieving near-linear time complexity.

Central to our analysis is the use of **Chebyshev's inequality**. While often introduced in purely statistical contexts, Chebyshev's inequality provides a useful probabilistic bound for ensuring that the sampled elements approximate the median accurately. In essence, it tells us

that $\mathbb{P}[|X - \mathbb{E}[X]| \geq k]$ can be kept small if $\mathrm{Var}[X]$ is finite, thereby giving us high confidence that our pivot-based "zoom-in" on the median is correct.

More concretely, we prove the following theorem with our algorithm.

Theorem 1.1 *There exists a randomized algorithm to compute the median that runs in time $\mathcal{O}(n)$ and succeeds with probability $1 - \mathcal{O}\left(n^{-1/4}\right)$.*

Thesis Outline

This thesis is structured as follows:

- **Chapter 3 (Notation and Preliminaries):** We define our notational conventions and offer a brief refresher on Chebyshev's inequality, setting the stage for the randomized analysis to come.
- **Chapter 4 (Algorithm and Analysis):** We detail the randomized median-finding algorithm, present its pseudocode, and provide a rigorous proof of correctness. We also analyze the runtime and show why it remains $\mathcal{O}(n)$ with high probability.
- **Chapter 5 (Conclusion):** We summarize our findings, discuss the algorithm's theoretical and practical implications, and highlight possible directions for future research.

Chapter 2

Literature Review

In this chapter, we review the relevant literature on multi-currency portfolio optimization. We discuss the contributions and methodologies in the field, highlighting how our work builds upon and extends existing research.

The core of our analysis builds on two recent studies that apply regularization techniques to international asset allocation. Burkhardt and Ulrych (2023) introduce a joint optimization approach within a mean-variance framework. By using L1 and L2 regularization, they show that jointly optimizing assets and currencies leads to better out-of-sample performance than the standard industry practice of using a separate currency overlay.

Extending this framework, Lucescu and Ulrych (2026) consider an Expected Shortfall optimization framework to better manage downside tail risk. To fix the numerical instability common in Expected Shortfall minimization, they apply similar regularization methods. Their findings reveal a dimensional trade-off. Joint optimization works best in small asset universes, but the separate currency overlay approach becomes more effective for larger universes due to its built-in dimensionality reduction.

Reproducing and comparing these two baseline frameworks serves as the starting point for the extensions proposed in this thesis.

Chapter 3

Notation and Preliminaries

Definitions and Notation

Arrays and Median

Throughout this thesis, we denote the input array of size n by

$$a = (a[1], a[2], \ldots, a[n]).$$

We may also use $a[1 \ldots n]$ to denote an array a of length n. Our goal is to find the *median* of a. Formally, we define:

- If n is odd, the median is the element in the $\lceil n/2 \rceil$-th position of the array when sorted in non-decreasing order.
- If n is even, the median is the element in the $\lceil n/2 \rceil$-th position of the sorted array.

Random Variables and Expectation

In the analysis of our randomized median-finding algorithm, we shall frequently refer to *random variables*. A random variable X is a function from the sample space Ω (representing all possible outcomes of a probabilistic experiment) to the real numbers $\mathbb{R}$.

Expectation. The *expected value*, or *mean*, of X is defined by

$$\mathbb{E}[X] = \sum_{\omega \in \Omega} X(\omega)\, \mathbb{P}[\omega],$$

in the discrete case. If X has a probability density function f_X (in the continuous case), then

$$\mathbb{E}[X] = \int_{-\infty}^{\infty} x\, f_X(x)\, dx.$$

Intuitively, $\mathbb{E}[X]$ indicates the "average" or "central" value of the random variable over many independent trials.

Variance and Standard Deviation

To measure how much a random variable deviates from its mean, we use its *variance*, denoted $\mathrm{Var}[X]$. This is defined as:

$$\mathrm{Var}[X] = \mathbb{E}[(X - \mathbb{E}[X])^2].$$

The *standard deviation* of X, denoted σ_X, is the square root of the variance:

$$\sigma_X = \sqrt{\mathrm{Var}[X]}.$$

A smaller variance (or standard deviation) means the random variable's values tend to lie closer to its mean, whereas a larger variance indicates a broader spread of values.

Asymptotic Notation

In describing the time complexity of algorithms, we use $\mathcal{O}(\ldots)$ notation in a standard way:

$$f(n) = \mathcal{O}(g(n)) \quad \Longleftrightarrow \quad \lim_{n\to\infty} \frac{f(n)}{g(n)} < \infty.$$

We may also use $\Theta(\ldots)$, $\Omega(\ldots)$, and other asymptotic notations where necessary.

Chebyshev's Inequality

Chebyshev's inequality is a cornerstone result in probability theory, providing a quantitative bound on how a random variable deviates from its expected value. Formally, let X be any random variable with finite mean $\mathbb{E}[X]$ and finite variance $\mathrm{Var}[X]$. Then, for any $k > 0$,

$$\mathbb{P}[|X - \mathbb{E}[X]| \geq k] \;\leq\; \frac{\mathrm{Var}[X]}{k^2}.$$

Intuition and Relation to Markov's Inequality

To build intuition, recall the simpler *Markov's inequality*, which states that for any nonnegative random variable Y and $t > 0$,

$$\mathbb{P}[Y \geq t] \leq \frac{\mathbb{E}[Y]}{t}.$$

Chebyshev's inequality is often viewed as an application of Markov's inequality to the random variable $(X - \mathbb{E}[X])^2$. Indeed,

$$\mathbb{P}[(X - \mathbb{E}[X])^2 \geq k^2] \leq \frac{\mathbb{E}[(X - \mathbb{E}[X])^2]}{k^2} = \frac{\mathrm{Var}[X]}{k^2},$$

and from $\{\, |X - \mathbb{E}[X]| \geq k \,\} \equiv \{\, (X - \mathbb{E}[X])^2 \geq k^2 \,\}$, we obtain Chebyshev's inequality.

The key takeaway is that Chebyshev's inequality captures a broad class of distributions. It requires no assumption of normality or other specific distributional shapes, only finite variance. In exchange for this generality, Chebyshev's bound can be somewhat loose compared to more specialized inequalities (e.g., Hoeffding's, Chernoff's, or Bernstein's). Nonetheless, it remains a powerful and classic tool for bounding "tail events," i.e. events in which X significantly deviates from its mean.

Why We Use Chebyshev's Inequality Here

In our randomized median-finding algorithm, we draw random samples from the input array and use them to approximate the true median. The "rank" of our chosen pivots in the sorted order of the entire array are themselves random variables. We want to show that, with high probability, these rank containt the $\lceil n/2 \rceil$-th position.

By applying Chebyshev's inequality, we obtain a bound on $\mathbb{P}[|X - \mathbb{E}[X]| \geq k]$ for a well-chosen k, where X is the rank of our pivot. In other words, Chebyshev's inequality formally establishes

the reliability of our sampling strategy by demonstrating that, except with small probability, our pivots partition the array around the actual median. This allows us to restrict further computation to a subset of the array whose size is guaranteed to be sufficiently small, thereby ensuring linear running time.

Extensions and Alternatives

While Chebyshev's inequality is sufficient for our purposes, more refined analyses might employ other concentration bounds such as Hoeffding's or Chernoff's inequalities. These stronger bounds can yield tighter probabilities when the underlying sample distribution has additional regularity properties (e.g., bounded random variables or independent Bernoulli trials). However, Chebyshev's inequality has the advantage of simplicity and broad applicability, making it an ideal choice for illustrating the core ideas in a clear manner.

Chapter 4

Algorithm and Analysis

In this chapter, we present our main randomized median-finding algorithm in full detail and analyze its performance. We first give a high-level description and provide the pseudocode in Algorithm 1. We then prove two central theorems: one that establishes the algorithm's probability of success (i.e. returning the correct median) and another that bounds its running time by $\mathcal{O}(n)$. Throughout this chapter, we use the notation introduced in Chapter 3.

Algorithm Description

The guiding idea behind the algorithm is to select a "small" random sample from the input array, sort this sample, and use carefully chosen elements from this sample as *pivots* to bracket the true median. Specifically:

- We draw $m = \lceil n^{3/4} \rceil$ elements uniformly at random from the input array $a[1 \dots n]$ and sort these sampled elements into an array $b[1 \dots m]$.
- We then pick two elements from b, denoted p_ℓ and p_r, near the middle of b but offset by $\pm\sqrt{n}$. Intuitively, these two pivots should "sandwich" the true median of the entire array with high probability.
- We gather all elements of the original array a that lie between these two pivots (inclusive) into a temporary array T. We verify that the median must lie in this sub-array, and that T is not excessively large.
- If those checks are satisfied, we sort T and extract the median; otherwise, the algorithm reports a *failure*.

The rest of this chapter gives a proof of two properties of this algorithm:

1. With high probability, Algorithm 1 does not fail and returns the correct median.
2. Conditioned on success, the overall running time is $\mathcal{O}(n)$.

Probability of Success

We begin by showing that, except with small probability, the two pivots p_ℓ and p_r are chosen so that the sub-array T contains the true median of a and is not excessively large. Formally, we have:

Theorem 4.1 (Correctness with High Probability) *The probability that Algorithm 1 returns the correct median is at least* $1 - \mathcal{O}\left(n^{-1/4}\right)$. *In other words, the algorithm succeeds* with high probability.

Algorithm 1 RandMedian: Randomized Median Algorithm

Require: Unsorted array $a[1 \dots n]$ of size n.
Ensure: Median of a or *failure*.

Initialize an empty array b.
For each element $x \in a[1 \dots n]$:
  Include x in b **with probability** $n^{-1/4}$, independently of other elements.
$m \leftarrow |b|$ ▷ (Random) size of the sample; note that $\mathbb{E}[m] = n^{3/4}$.
if $m > n^{3/4} + \sqrt{n}$ **then**
  return *failure*
end if
Sort b in $\mathcal{O}(m \log m)$ time
Compute pivots:

$$p_\ell \leftarrow b\left[\left\lfloor \frac{m}{2} - \sqrt{n} \right\rfloor\right], \quad p_r \leftarrow b\left[\left\lceil \frac{m}{2} + \sqrt{n} \right\rceil\right]$$

Define the subset $T \leftarrow \{\, x \in a : p_\ell \leq x \leq p_r \}$
Compute:

$$s \leftarrow \#\{\, x \in a : x < p_\ell \},$$
$$t \leftarrow \#\{\, x \in a : p_\ell \leq x \leq p_r \}.$$

if $s < \lceil n/2 \rceil$ **and** $s + t \geq \lceil n/2 \rceil$ **and** $t < 4n^{3/4}$ **then**
  Sort T in $\mathcal{O}(|T| \log |T|)$ time
  Median of $a \leftarrow T[\lceil n/2 \rceil - s]$
  return Median of a
else
  return *failure*
end if

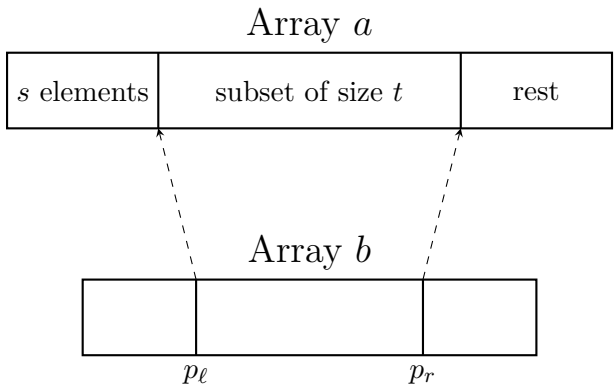

Figure 4.1: Illustration of the arrays and the pivots, showing array a, sampled array b, and the pivots p_ℓ, p_r. Note that this visualization shows the sorted version of the arrays.

Proof For simplicity, we assume throughout this proof that n is even and that $n^{3/4}$ is an integer; this avoids cumbersome floor and ceiling functions. The general case follows by analogous, but slightly more tedious, arguments.

Recall that $a[\,1\ldots n\,]$ is our input array of n distinct elements. Let $b[\,1\ldots m\,]$ be the sample of size m drawn by sampling each element with probability $n^{-1/4}$ from a. The setup is also visualized in Figure 4.1. We then sort b, pick the pivots[1]

$$p_\ell \;=\; b\Big[\,\tfrac{m}{2} - \sqrt{n}\Big], \quad p_r \;=\; b\Big[\,\tfrac{m}{2} + \sqrt{n}\Big],$$

and collect into T all elements $x \in a$ with $p_\ell \le x \le p_r$. Let

$$s \;=\; \#\{\,x \in a : x < p_\ell\,\} \quad \text{and} \quad t \;=\; \#\{\,x \in a : p_\ell \le x \le p_r\,\}.$$

Key sets and random variables. For any integer $k \in \{0, \ldots, n\}$, define

$$S_k \;=\; \{\text{the } k \text{ smallest elements of } a\}.$$

We then define the random variable

$$X_k \;=\; \#\{\,x \in b : x \in S_k\}.$$

Since each element of a is independently included in b with probability $n^{-1/4}$, the number of elements of S_k that appear in b follows

$$X_k \;\sim\; \mathrm{Bin}\big(k,\, n^{-1/4}\big).$$

Hence,

$$\mathbb{E}[X_k] \;=\; k\,n^{-1/4}, \quad \mathrm{Var}[X_k] \;=\; k\,n^{-1/4}\big(1 - n^{-1/4}\big) \;\le\; k\,n^{-1/4}.$$

Application of Chebyshev's Inequality. Since $X_k \sim \mathrm{Bin}(k,\, n^{-1/4})$ with $\mathrm{Var}[X_k] \le k\,n^{-1/4}$, Chebyshev's inequality gives

$$\mathbb{P}[|X_k - k\,n^{-1/4}| \;\ge\; \sqrt{n}] \;\le\; \frac{\mathrm{Var}[X_k]}{n} \;\le\; \frac{k\,n^{-1/4}}{n} \;=\; \frac{k}{n^{5/4}}. \tag{4.1}$$

Defining the "bad" events. We show that the only way the algorithm can fail (i.e. not return the correct median) is if one of the following four events occurs. Denote

$$\begin{aligned}
A \;&:=\; \{\,s \;\ge\; n/2\}\,,\\
B \;&:=\; \{\,s + t < n/2\}\,,\\
C \;&:=\; \{\,s < \tfrac{n}{2} - 2\,n^{3/4}\}\,,\\
D \;&:=\; \{\,s + t \;\ge\; \tfrac{n}{2} + 2\,n^{3/4}\}\,.\\
E \;&:=\; \{\,m > n^{3/4} + \sqrt{n}\}\,.
\end{aligned}$$

Notice that E not occuring means that the first "return failure" line is not executed. If none of these events happens, then we must have

$$s < \tfrac{n}{2}, \quad s + t \;\ge\; \tfrac{n}{2}, \quad s \;\ge\; \tfrac{n}{2} - 2\,n^{3/4}, \quad \text{and} \quad s + t < \tfrac{n}{2} + 2\,n^{3/4}.$$

One can check that these conditions imply exactly

$$\big(s < \tfrac{n}{2}\big) \quad \text{and} \quad \big(s + t \ge \tfrac{n}{2}\big) \quad \text{and} \quad t < 4\,n^{3/4},$$

so the algorithm's final check (line 11 of the pseudocode) succeeds. In that case, our pivot-based bracket $[\,p_\ell,\, p_r\,]$ indeed contains the median, and the algorithm returns the correct value.

[1] Technically, if m is too small, the following lines are undefined (out of bounds). But this is avoided whp, as for the other events later in the proof.

Probability of each bad event. We next prove that $\mathbb{P}[A], \mathbb{P}[B], \mathbb{P}[C], \mathbb{P}[D], \mathbb{P}[E]$ are each at most $n^{-1/4}$. Observe:

- Event A means $s \geq n/2$. Hence among the $n/2$ smallest elements of a (i.e. the set $S_{n/2}$), at most $\frac{1}{2}\, n^{3/4} - \sqrt{n}$ of them are drawn into b. In other words,

$$X_{n/2} \;\leq\; \tfrac{1}{2}\, n^{3/4} \;-\; \sqrt{n}.$$

  But from $\mathbb{E}[X_{n/2}] = (n/2)\, n^{-1/4} = \frac{1}{2}\, n^{3/4}$ and (4.1), we get $\mathbb{P}[X_{n/2} \leq \frac{1}{2}\, n^{3/4} - \sqrt{n}] \leq n^{-1/4}$. Thus $\mathbb{P}[A] \leq n^{-1/4}$.

- Event B means $s + t < n/2$. Equivalently, the set $S_{n/2}$ has more than $\frac{1}{2}\, n^{3/4} + \sqrt{n}$ elements contained in b. Hence

$$X_{n/2} \;\geq\; \tfrac{1}{2}\, n^{3/4} \;+\; \sqrt{n}.$$

  Again, from $\mathbb{E}[X_{n/2}] = \frac{1}{2}\, n^{3/4}$ and (4.1), we get $\mathbb{P}[X_{n/2} \geq \frac{1}{2}\, n^{3/4} + \sqrt{n}] \leq n^{-1/4}$. So $\mathbb{P}[B] \leq n^{-1/4}$.

- Event C means $s < n/2 - 2\, n^{3/4}$. In this case, the set $S_{n/2-2\, n^{3/4}}$ must have at least $\frac{1}{2}\, n^{3/4} - \sqrt{n}$ of its elements in b. Formally,

$$X_{n/2-2\, n^{3/4}} \;\geq\; \tfrac{1}{2}\, n^{3/4} - \sqrt{n}.$$

  Using $\mathbb{E}[X_{n/2-2\, n^{3/4}}] = (n/2 - 2\, n^{3/4})\, n^{-1/4} = \frac{1}{2} n^{3/4} - 2\sqrt{n}$ and (4.1) again shows $\mathbb{P}[C] \leq n^{-1/4}$.

- Event D means $s + t \geq n/2 + 2\, n^{3/4}$. Hence the set $S_{n/2+2\, n^{3/4}}$ contains at most $\frac{1}{2}\, n^{3/4} + \sqrt{n}$ elements from b. That is,

$$X_{n/2+2\, n^{3/4}} \;\leq\; \tfrac{1}{2}\, n^{3/4} \;+\; \sqrt{n}.$$

  A final application of (4.1) yields $\mathbb{P}[D] \leq n^{-1/4}$.

- A similar calculation involving X_n shows that $\mathbb{P}[E] \leq n^{-1/4}$.

Conclusion. By the union bound,

$$\mathbb{P}[A \cup B \cup C \cup D \cup E] \;\leq\; \mathbb{P}[A] + \mathbb{P}[B] + \mathbb{P}[C] + \mathbb{P}[D] + \mathbb{P}[E] \;\leq\; 5 \times n^{-1/4}$$

In other words, with probability at least $1 - 5n^{-1/4}$, *none* of these events occurs. When all five do not occur, the algorithm's checks

$$s < \lceil n/2 \rceil, \quad s + t \geq \lceil n/2 \rceil, \quad t < 4\, n^{3/4}$$

are satisfied, so the algorithm terminates successfully with the correct median. Hence the overall success probability is at least $1 - 5n^{-1/4}$, completing the proof. □

Runtime Analysis

Next, we establish that Algorithm 1 achieves linear time complexity *with high probability.* Note that if the algorithm returns *failure*, it may be retried or combined with a fallback procedure. Here, we focus on the typical execution that succeeds.

Theorem 4.2 (Runtime) *Conditioned on the event that Algorithm 1 does not fail (an event which, by Theorem 4.1, holds with high probability), the total running time is $\mathcal{O}(n)$.*

Proof We break the cost into four parts:

1. **Sampling and Sorting b.** We pick $m \leq n^{3/4} + \sqrt{n} \leq 2n^{3/4}$ elements randomly from a (conditioned on success). This is done in $\mathcal{O}(n)$ time. Sorting b of size m then takes

$$\mathcal{O}(m \log(m)) = \mathcal{O}\Big(n^{3/4} \log\Big(n^{3/4}\Big)\Big).$$

   Since $n^{3/4} \log\Big(n^{3/4}\Big)$ is asymptotically smaller than n for large n, this component is $o(n)$.

2. **Computing Ranks and Sub-array T.** We determine how many elements of a lie below p_ℓ, how many lie above p_r, etc. This amounts to a single pass through a, comparing each element of a to p_ℓ and p_r. Hence, it costs $\mathcal{O}(n)$ time.

3. **Sorting T.** By Algorithm 1, we only proceed to sort T if $t < 4n^{3/4}$. Thus,

$$t \leq 4n^{3/4}.$$

   Sorting T via a standard comparison-based algorithm costs

$$\mathcal{O}(|T| \log(|T)|) \leq \mathcal{O}\Big(n^{3/4} \log\Big((n^{3/4})\Big)\Big),$$

   which again is $o(n)$.

4. **Extracting the Median.** After sorting T, we simply pick the element of T whose rank corresponds to $\lceil n/2 \rceil - s$. This is clearly an $\mathcal{O}(1)$ operation.

Summing these components, the *dominant term* is $\mathcal{O}(n)$ (from the pass to identify s and t). The sublinear sorting costs on b and on T both constitute $\mathcal{O}\Big(n^{3/4} \log(n)\Big)$, which is strictly smaller than n for large n. Consequently, the total running time is $\mathcal{O}(n)$. □

In conclusion, the previous two theorems immediately prove Theorem 1.1, our main result, which we restate here for convenience.

Theorem 1.1 *There exists a randomized algorithm to compute the median that runs in time $\mathcal{O}(n)$ and succeeds with probability $1 - \mathcal{O}\Big(n^{-1/4}\Big)$.*

Discussion and Extensions

Theorems 4.1 and 4.2 jointly show that Algorithm 1 achieves $\mathcal{O}(n)$ time with high probability of success. In rare cases where the check fails, the algorithm returns *failure* rather than risking an incorrect answer. One can either re-run the algorithm (the probability of repeated failures being very low) or combine it with a fallback deterministic linear-time median-finding routine such as the median-of-medians approach. The randomized algorithm typically attains lower hidden constants in practice than purely deterministic linear-time solutions, thereby offering an attractive balance of theoretical guarantees and real-world performance.

In the next chapter, we summarize our results and highlight possible directions for future work on randomized selection algorithms.

Chapter 5

Conclusion

In this thesis, we presented a randomized selection algorithm for identifying the median of an unsorted array in expected linear time. Our method hinges on sampling a sublinear number of elements, sorting them to obtain pivots, and then "zooming in" on a small region of the array that is guaranteed, with high probability, to contain the true median. The crux of our analysis relied on Chebyshev's inequality, which provided a principled way to bound the probability that our random pivots deviate significantly from the actual median's rank in the sorted order.

Summary of Contributions.

- We introduced a randomized median-finding algorithm that leverages a sample of size roughly $n^{3/4}$. Through sorting this sample and selecting pivots near its middle, the algorithm narrows down the search space to a sub-array whose size is no more than $4n^{3/4}$.
- We proved a high-probability bound (Theorem 4.1) on the correctness of the algorithm. Specifically, with probability at least $1 - \mathcal{O}\left(n^{-1/4}\right)$, our pivots capture the true median, ensuring that the algorithm does not report *failure*.
- We established a near-linear worst-case running time (Theorem 4.2). The cost of sorting the small sample and, conditionally, the sub-array T both remain asymptotically lower than $\mathcal{O}(n)$, leaving the dominant cost to be a single pass over the array to compute element ranks relative to the pivots. Hence, the overall algorithm runs in $\mathcal{O}(n)$ time with high probability.
- Beyond the immediate theoretical guarantees, we discussed how to combine our randomized approach with a deterministic "median-of-medians" fallback to handle rare failure events, ensuring correctness remains absolute. This hybrid strategy preserves the practical advantages of randomization while delivering a worst-case linear-time bound.

Limitations and Possible Extensions. While our use of Chebyshev's inequality is conceptually simple and broadly applicable, tighter concentration results (e.g. Chernoff or Hoeffding bounds) could potentially yield smaller failure probabilities or allow an even smaller sample size. However, such refinements might require more specific assumptions about the distribution of the input data or the independence structure of the sampling process.

Another direction for future exploration is to extend the approach to streaming and dynamic settings. In scenarios where the dataset arrives incrementally, maintaining a small, representative sample to approximate the median could be achieved through reservoir sampling or related techniques. Additionally, one might investigate parallel or distributed variants of the

algorithm, where the sampling and pivot-selection steps are split across multiple processors or machines.

Chapter 6

Model-free multi-currency portfolio returns

In this chapter, we derive a model-free representation of one-period portfolio returns for an investor who holds assets denominated in multiple currencies and can take currency forward positions for hedging. The derivations follow the methodology presented in [**?**] and [**?**], using simple returns and assume that rebalancing occurs at the end of each period. The representation is used in subsequent chapters to formulate and analyze the results of the multi-currency portfolio optimization problem.

6.1 Unhedged Portfolio Returns

We consider an international investor that allocates his wealth across N assets denominated in up to $M+1$ currencies. We fix a discrete time index $t \in \{0, 1, 2, \dots\}$ and one investment period from t to $t{+}1$. Let the investor's *domestic currency* be indexed by 0 and let the rest of the M foreign currencies be indexed by $\mathcal{C} \coloneqq \{1, 2, \dots, M\}$.

Let $P_{i,t} > 0$ be the price of asset i at time t in its local currency $c_i \in \mathcal{C} \cup \{0\}$. Define the corresponding *simple local currency asset return* over $(t, t{+}1]$ as

$$R^{\mathrm{LC}}_{i,t+1} \coloneqq \frac{P_{i,t+1} - P_{i,t}}{P_{i,t}}.$$

Thus $P_{i,t+1} = P_{i,t}(1 + R^{\mathrm{LC}}_{i,t+1})$.

For each foreign currency $c \in \mathcal{C}$, let $S_{c,t} > 0$ denote the exchange rate at time t, quoted as units of domestic currency per one unit of foreign currency c. The associated *simple currency return* over $(t, t{+}1]$ is

$$e_{c,t+1} \coloneqq \frac{S_{c,t+1} - S_{c,t}}{S_{c,t}}.$$

Equivalently, $S_{c,t+1} = S_{c,t}(1 + e_{c,t+1})$. By definition for the domestic currency, we set $S_{0,t} \equiv 1$ and $e_{0,t+1} \equiv 0$.

The domestic currency value of holding one unit of asset i at time t is $P_{i,t}\, S_{c_i,t}$. Therefore, the *unhedged domestic currency simple return* of asset i is

$$\begin{aligned} R^{u}_{i,t+1} &= \frac{P_{i,t+1} S_{c_i,t+1} - P_{i,t} S_{c_i,t}}{P_{i,t} S_{c_i,t}} \\ &= \frac{P_{i,t}(1 + R^{\mathrm{LC}}_{i,t+1}) S_{c_i,t}(1 + e_{c_i,t+1}) - P_{i,t} S_{c_i,t}}{P_{i,t} S_{c_i,t}} \\ &= R^{\mathrm{LC}}_{i,t+1} + e_{c_i,t+1} + R^{\mathrm{LC}}_{i,t+1}\, e_{c_i,t+1}. \end{aligned} \tag{6.1}$$

Let $\boldsymbol{x}_t \in \mathbb{R}^N$ be the vector of asset portfolio weights chosen at time t and held until $t{+}1$, where $x_{i,t}$ is the fraction of total portfolio value invested in asset i at time t. We impose the standard budget constraint $\mathbf{1}^\top \boldsymbol{x}_t = 1$.

Assuming no rebalancing between times t and $t{+}1$, the domestic currency portfolio's unhedged return is given by

$$R^u_{\mathcal{P},t+1} = \sum_{i=1}^{N} x_{i,t}\, R^u_{i,t+1}.$$

6.2 Hedged Portfolio Returns

To manage currency risk, the investor can take positions in one-period currency forward contracts. A forward contract on foreign currency c entered at time t settles at time $t{+}1$ with payoff (in domestic currency) of $F_{c,t} - S_{c,t+1}$, where $F_{c,t}$ is the forward rate quoted at time t. The forward rate is typically set such that the contract has zero value at inception, i.e. $F_{c,t} = S_{c,t}(1+r_t)/(1+r_{c,t})$, where r_t and $r_{c,t}$ are the domestic and foreign risk-free rates, respectively. The *excess currency return* over the period is $e_{c,t+1} - f_{c,t}$, where $f_{c,t} = (F_{c,t} - S_{c,t})/S_{c,t}$ is the forward premium. The investor can choose the forward position $\tau_{c,t}$ (expressed as a fraction of total portfolio value) for each foreign currency c to hedge against currency risk. The *hedged* portfolio return is then the sum of the unhedged asset returns and the payoffs from the forward positions:

$$R^h_{\mathcal{P},t+1} = \sum_{i=1}^{N} x_{i,t}\, R^u_{i,t+1} + \sum_{c=1}^{M} \tau_{c,t}\,(f_{c,t} - e_{c,t+1}).$$

For each foreign currency $c \in \mathcal{C}$, assume that at time t the investor can enter a one-period currency forward contract that settles at $t{+}1$, with forward rate $F_{c,t}$ quoted in domestic currency per unit of foreign currency c. Define the *forward premium* (simple rate) by

$$f_{c,t} := \frac{F_{c,t} - S_{c,t}}{S_{c,t}}.$$

Let $\boldsymbol{\tau}_t \in \mathbb{R}^M$ denote forward positions expressed as fractions of total portfolio value, where $\tau_{c,t}$ represents the relative notional of the *short* forward in currency c at time t. With this sign convention, $\tau_{c,t} > 0$ represents hedging (shorting foreign currency forward). The *hedged* portfolio return is

$$R^h_{P,t+1} = \sum_{i=1}^{N} x_{i,t}\, R^u_{i,t+1} + \sum_{c=1}^{M} \tau_{c,t}\,(f_{c,t} - e_{c,t+1}).$$

This representation is model-free: it holds pathwise for realized returns and does not assume any distributional structure.

6.3 Matrix Representation and Decomposition

To express the returns in a compact form suitable for optimization, we define the currency incidence matrix $\boldsymbol{C} \in \{0,1\}^{N\times M}$ by

$$C_{i,c} := \begin{cases} 1, & \text{if asset } i \text{ is denominated in foreign currency } c, \\ 0, & \text{otherwise.} \end{cases}$$

The portfolio's *implicit foreign-currency exposure* is $\boldsymbol{w}_t := \boldsymbol{C}^\top \boldsymbol{x}_t \in \mathbb{R}^M$, where $w_{c,t} = \sum_{i:\, c_i = c} x_{i,t}$.

Collect returns into vectors $\boldsymbol{R}_{t+1}^{\mathrm{LC}} \in \mathbb{R}^N$, $\boldsymbol{e}_{t+1} \in \mathbb{R}^M$, and the forward premiums $\boldsymbol{f}_t \in \mathbb{R}^M$. Define the currency-return vector aligned to assets via $\boldsymbol{C}\boldsymbol{e}_{t+1} \in \mathbb{R}^N$, where the ith component is $e_{c_i,t+1}$ for foreign assets and 0 for domestic assets. Using (6.1), the vector of *unhedged domestic returns* across assets is

$$\boldsymbol{R}_{t+1}^{u} = \boldsymbol{R}_{t+1}^{\mathrm{LC}} + \boldsymbol{C}\boldsymbol{e}_{t+1} + (\boldsymbol{R}_{t+1}^{\mathrm{LC}} \odot \boldsymbol{C}\boldsymbol{e}_{t+1}),$$

where $\odot$ denotes the Hadamard product.

Bibliography

[1] Manuel Blum, Robert W Floyd, Vaughan R Pratt, Ronald L Rivest, and Robert E Tarjan. Time bounds for selection. *Journal of Computer and System Sciences*, 7(4):448–461, 1973.

Appendices

ES Optimization Equivalence

We derive the equivalence between the direct minimization of Expected Shortfall and the auxiliary formulation proposed by Rockafellar and Uryasev (2000) that allows for formulating the optimization as a linear program.

We assume that the loss distribution is continuous, meaning the loss cumulative distribution function has no jumps and the probability density function $f(l)$ exists. Under these assumptions, Value-at-Risk (VaR) is uniquely defined.

Define the loss random variable at time $t+1$ given portfolio weights x_t as $L(x_t) = -x_t^T R_{t+1}$. Let $f(l)$ denote the probability density function of the loss $L(x_t)$. The VaR at confidence level β is defined as the β-quantile of the loss distribution

$$\mathrm{VaR}_\beta(x_t) = \min\{\alpha : \mathbb{P}(L(x_t) \leq \alpha) \geq \beta\}.$$

The Expected Shortfall is defined as the conditional expectation of losses exceeding VaR

$$\mathrm{ES}_\beta(x_t) = \mathbb{E}[L(x_t) \mid L(x_t) \geq \mathrm{VaR}_\beta(x_t)] = \frac{1}{1-\beta} \int_{\mathrm{VaR}_\beta(x_t)}^{\infty} l f(l)\, dl.$$

We now consider the auxiliary function F_β defined as

$$F_\beta(x_t, \alpha) = \alpha + \frac{1}{1-\beta} \mathbb{E}_t[(L(x_t) - \alpha)^+].$$

We can rewrite the expectation term using the integral form

$$\mathbb{E}_t[(L(x_t) - \alpha)^+] = \int_{-\infty}^{\infty} (l-\alpha)^+ f(l)\, dl = \int_{\alpha}^{\infty} (l-\alpha) f(l)\, dl.$$

Before minimizing $F_\beta(x_t, \alpha)$ with respect to α, we verify convexity. The second derivative of F_β with respect to α is

$$\begin{aligned} \frac{\partial F_\beta}{\partial \alpha} &= 1 - \frac{1}{1-\beta} \int_{\alpha}^{\infty} f(l)\, dl \\ \frac{\partial^2 F_\beta}{\partial \alpha^2} &= \frac{1}{1-\beta} f(\alpha). \end{aligned}$$

Since the probability density function $f(\alpha) \geq 0$ and $\beta < 1$, the second derivative is non-negative, $\frac{\partial^2 F_\beta}{\partial \alpha^2} \geq 0$. Thus, $F_\beta(x_t, \alpha)$ is convex in α, ensuring that the first-order condition is necessary and sufficient for a global minimum.

Differentiation of $F_\beta(x_t, \alpha)$ with respect to α yields the first-order condition for optimality. Using Leibniz's rule we have

$$\begin{aligned}
\frac{\partial F_\beta}{\partial \alpha} &= 1 + \frac{1}{1-\beta}\frac{\partial}{\partial \alpha}\int_\alpha^\infty (l-\alpha) f(l)\, dl \\
&= 1 + \frac{1}{1-\beta}\left[-f(\alpha)\cdot 0 + \int_\alpha^\infty (-1) f(l)\, dl\right] \\
&= 1 - \frac{1}{1-\beta}\int_\alpha^\infty f(l)\, dl \\
&= 1 - \frac{1}{1-\beta}\mathbb{P}(L(x_t) \geq \alpha).
\end{aligned}$$

Setting the derivative to zero to find the optimal α^*

$$\begin{aligned}
1 - \frac{1}{1-\beta}\mathbb{P}(L(x_t) \geq \alpha^*) &= 0 \\
\mathbb{P}(L(x_t) \geq \alpha^*) &= 1-\beta \\
\mathbb{P}(L(x_t) \leq \alpha^*) &= \beta.
\end{aligned}$$

This implies that the optimal α^* that minimizes the auxiliary function for a fixed x_t is exactly the Value-at-Risk at level β, so $\alpha^* = \mathrm{VaR}_\beta(x_t)$.

Substituting $\alpha^* = \mathrm{VaR}_\beta(x_t)$ back into $F_\beta(x_t, \alpha)$ we obtain

$$\begin{aligned}
F_\beta(x_t, \mathrm{VaR}_\beta(x_t)) &= \mathrm{VaR}_\beta(x_t) + \frac{1}{1-\beta}\int_{\mathrm{VaR}_\beta(x_t)}^\infty (l - \mathrm{VaR}_\beta(x_t)) f(l)\, dl \\
&= \mathrm{VaR}_\beta(x_t) + \frac{1}{1-\beta}\left(\int_{\mathrm{VaR}_\beta(x_t)}^\infty l f(l)\, dl - \mathrm{VaR}_\beta(x_t)\int_{\mathrm{VaR}_\beta(x_t)}^\infty f(l)\, dl\right).
\end{aligned}$$

Recognizing that $\int_{\mathrm{VaR}_\beta(x_t)}^\infty f(l)\, dl = \mathbb{P}(L(x_t) \geq \mathrm{VaR}_\beta(x_t)) = 1-\beta$, we have

$$\begin{aligned}
F_\beta(x_t, \mathrm{VaR}_\beta(x_t)) &= \mathrm{VaR}_\beta(x_t) + \frac{1}{1-\beta}\int_{\mathrm{VaR}_\beta(x_t)}^\infty l f(l)\, dl - \frac{1}{1-\beta}\mathrm{VaR}_\beta(x_t)(1-\beta) \\
&= \mathrm{VaR}_\beta(x_t) + \mathrm{ES}_\beta(x_t) - \mathrm{VaR}_\beta(x_t) \\
&= \mathrm{ES}_\beta(x_t).
\end{aligned}$$

Thus, minimizing $\mathrm{ES}_\beta(x_t)$ with respect to x_t is equivalent to minimizing $F_\beta(x_t, \alpha)$ jointly with respect to x_t and α

$$\min_{x_t} \mathrm{ES}_\beta(x_t) = \min_{x_t, \alpha}\left(\alpha + \frac{1}{1-\beta}\mathbb{E}_t[(-x_t^T R_{t+1} - \alpha)^+]\right).$$

Finally, we observe that $F_\beta(x_t, \alpha)$ is jointly convex in (x_t, α). The term α is linear (hence convex). The inner map $(x_t, \alpha) \mapsto (-x_t^\top R_{t+1} - \alpha)$ is affine. The outer function $z \mapsto z^+$ is convex and nondecreasing. The composition of a convex nondecreasing function with an affine map is convex. Thus $(-x_t^\top R_{t+1} - \alpha)^+$ is jointly convex in (x_t, α). Taking the expectation preserves convexity.

Since the objective function is convex, any local minimum is global, ensuring that optimization algorithms will converge efficiently to the optimal solution.

ES Optimization Implementation

Building upon the continuous equivalence derived in Appendix 6.3 and following the optimization framework in the original paper (Ulyrch, Lucescu 2026), this section details the precise matrix formulations required to solve the regularized multi-currency Expected Shortfall models. By linearizing the L_1 penalty and discretizing the ES computations, we map both the separate overlay and joint optimization frameworks into standard Quadratic Programs of the form

$$\min_{\tilde{z}} \left(\tilde{c}^T \tilde{z} + \frac{1}{2} \tilde{z}^T \tilde{Q} \tilde{z} \right) \quad \text{subject to} \quad \tilde{D}\tilde{z} = \tilde{E}, \quad \tilde{A}\tilde{z} \leq b, \quad \tilde{z} \geq 0, \tag{.2}$$

where $\tilde{z}$ is an augmented decision vector containing the portfolio weights and all necessary auxiliary variables.

Sample-Based Expected Shortfall

In practice, the conditional expectation from the ES objective is approximated using q historical joint return observations. Let $R_{\text{data}} \in \mathbb{R}^{q \times N}$ be the matrix of historical asset returns, where each row R_k^T represents the return vector at time k. The sample-based objective becomes:

$$\min_{x_t, \alpha} \left(-\mu^T x_t + \gamma\alpha + \frac{\gamma}{q(1-\beta)} \sum_{k=1}^{q} (-R_k^T x_t - \alpha)^+ \right).$$

To linearize the maximum operator $(\cdot)^+$, Rockafellar and Uryasev (2000) introduce a q-dimensional vector of auxiliary variables $u \in \mathbb{R}^q$, where $u_k = (-R_k^T x_t - \alpha)^+$. This implies two conditions for each sample k:

$$\begin{aligned} u_k &\geq 0 \\ u_k &\geq -R_k^T x_t - \alpha. \end{aligned}$$

Rearranging the second condition to isolate the variables yields a standard linear inequality:

$$-R_k^T x_t - \alpha - u_k \leq 0 \quad \forall\, k \in \{1, \ldots, q\}. \tag{.3}$$

In block matrix notation across all q samples, this becomes:

$$-R_{\text{data}} x_t - \mathbf{1}_q \alpha - I_q u \leq \mathbf{0}_q,$$

where $\mathbf{1}_q$ is a q-dimensional vector of ones, I_q is the $q \times q$ identity matrix, and $\mathbf{0}_q$ is a q-dimensional vector of zeros. This constructs the first three block columns of the general inequality constraint matrix $\tilde{A}$.

Linearization of the L_1 Penalty and Leverage

The L_1 norm $\|x_t\|_1 = \sum_{i=1}^{N} |x_{i,t}|$ introduces non-differentiability into the objective function. To resolve this, we introduce non-negative auxiliary variables representing the positive and negative components of each asset weight: $\Delta_t^+, \Delta_t^- \in \mathbb{R}^N$ such that $\Delta_t^+ \geq 0$ and $\Delta_t^- \geq 0$.

The weights are decomposed as:

$$x_t = \Delta_t^+ - \Delta_t^- \implies I_N x_t + I_N \Delta_t^- - I_N \Delta_t^+ = \mathbf{0}_N. \tag{.4}$$

Because the optimization minimizes the penalties, at most one of $\Delta_{i,t}^+$ or $\Delta_{i,t}^-$ will be non-zero for any asset i. Therefore, the absolute value is exactly determined by their sum: $|x_{i,t}| = \Delta_{i,t}^+ + \Delta_{i,t}^-$.

This allows the L_1 penalty term to be formulated linearly in the objective function as $\lambda_1 \mathbf{1}_N^T \Delta_t^+ + \lambda_1 \mathbf{1}_N^T \Delta_t^-$. Furthermore, the portfolio leverage constraint $\|x_t\|_1 \leq \ell$ transforms into a linear inequality:

$$\mathbf{1}_N^T \Delta_t^+ + \mathbf{1}_N^T \Delta_t^- \leq \ell. \tag{.5}$$

Quadratic Translation of the L_2 Penalty

Unlike the expected shortfall and L_1 terms which map to the linear objective vector $\tilde{c}$, the L_2 penalty integrates naturally into the quadratic matrix $\tilde{Q}$. The penalty is defined as:

$$\lambda_2 \|x_t\|_2^2 = \lambda_2 x_t^T x_t = \frac{1}{2} x_t^T (2\lambda_2 I_N) x_t.$$

Thus, the block of $\tilde{Q}$ corresponding to the weights x_t is populated with the diagonal matrix $2\lambda_2 I_N$, while all auxiliary variables receive zeros.

Separate Optimization (Currency Overlay)

In the separate approach, asset weights $x_t \in \mathbb{R}^N$ and currency exposures $\psi_t \in \mathbb{R}^M$ are optimized sequentially.

Step 1: Asset-Level Optimization

The target vector relies on fully hedged asset returns $R^{\text{fh}} \in \mathbb{R}^{q \times N}$. We define the augmented vector of dimension $(3N + q + 1)$:

$$\tilde{x}_t = \left(x_t^T, \quad \alpha, \quad u^T, \quad (\Delta_t^-)^T, \quad (\Delta_t^+)^T \right)^T.$$

The objective vector $\tilde{c}$ and quadratic matrix $\tilde{Q}$ are constructed as:

$$\tilde{c} = \begin{pmatrix} -\mu_{\text{fh}} \\ \gamma \\ \frac{\gamma}{q(1-\beta)} \mathbf{1}_q \\ \lambda_1^a \mathbf{1}_N \\ \lambda_1^a \mathbf{1}_N \end{pmatrix}, \quad \tilde{Q} = \begin{pmatrix} 2\lambda_2^a I_N & O \\ O & O \end{pmatrix},$$

where O represents zero matrices of appropriate dimensions. The equality constraints enforce the budget condition $\mathbf{1}_N^T x_t = 1$ and the variable decomposition from Equation .4:

$$\tilde{D} = \begin{pmatrix} \mathbf{1}_N^T & 0 & \mathbf{0}_q^T & \mathbf{0}_N^T & \mathbf{0}_N^T \\ I_N & \mathbf{0}_N & O_{N \times q} & I_N & -I_N \end{pmatrix}, \quad \tilde{E} = \begin{pmatrix} 1 \\ \mathbf{0}_N \end{pmatrix}.$$

The inequality matrix $\tilde{A}$ and bound vector b encode the shortfall losses from Equation .3 and the leverage limit from Equation .5:

$$\tilde{A} = \begin{pmatrix} -R^{\text{fh}} & -\mathbf{1}_q & -I_q & O_{q\times N} & O_{q\times N} \\ \mathbf{0}_N^T & 0 & \mathbf{0}_q^T & \mathbf{1}_N^T & \mathbf{1}_N^T \end{pmatrix}, \quad b = \begin{pmatrix} \mathbf{0}_q \\ \ell \end{pmatrix}.$$

Step 2: Currency-Level Optimization

Conditional on the optimal asset weights x_t^*, the currency exposures ψ_t are optimized. Using historical excess currency returns $R^c \in \mathbb{R}^{q\times M}$, the augmented vector becomes $\tilde{\psi}_t \in \mathbb{R}^{(3M+q+1)}$.

Because currency overlays act as zero-investment portfolios, there is no budget constraint. Additionally, the pre-fixed asset portfolio returns ($R^{\text{fh}} x_t^*$) must be subtracted from the shortfall inequality constraint boundaries. Thus, the formulations shift to:

$$\tilde{D}_\psi = \begin{pmatrix} I_M & \mathbf{0}_M & O_{M\times q} & I_M & -I_M \end{pmatrix}, \quad \tilde{E}_\psi = \mathbf{0}_M,$$

$$\tilde{A}_\psi = \begin{pmatrix} -R^c & -\mathbf{1}_q & -I_q & O_{q\times M} & O_{q\times M} \end{pmatrix}, \quad b_\psi = R^{\text{fh}} x_t^*.$$

Joint Optimization Framework

The joint formulation merges assets and currencies into a single decision vector $\theta_t = (x_t^T, \psi_t^T)^T \in \mathbb{R}^{N+M}$, evaluated over the joint return block matrix $R^{\text{all}} = \begin{pmatrix} R^{\text{fh}} & R^c \end{pmatrix} \in \mathbb{R}^{q\times(N+M)}$.

We construct an expansive augmented vector $\tilde{\theta}_t \in \mathbb{R}^{(3N+3M+q+1)}$:

$$\tilde{\theta}_t = \begin{pmatrix} x_t^T, & \psi_t^T, & \alpha, & u^T, & (\Delta_{x,t}^-)^T, & (\Delta_{\psi,t}^-)^T, & (\Delta_{x,t}^+)^T, & (\Delta_{\psi,t}^+)^T \end{pmatrix}^T.$$

The objective vector applies separate L_1 penalty parameters (λ_1^a for assets, λ_1^c for currencies):

$$\tilde{c} = \begin{pmatrix} -\mu_{\text{fh}} \\ -\mu_c \\ \gamma \\ \frac{\gamma}{q(1-\beta)}\mathbf{1}_q \\ \lambda_1^a \mathbf{1}_N \\ \lambda_1^c \mathbf{1}_M \\ \lambda_1^a \mathbf{1}_N \\ \lambda_1^c \mathbf{1}_M \end{pmatrix}, \quad \tilde{Q} = \begin{pmatrix} 2\lambda_2^a I_N & O & O \\ O & 2\lambda_2^c I_M & O \\ O & O & O \end{pmatrix}.$$

The budget constraint applies strictly to the asset weights, while the decomposition constraints apply to both assets and currencies.

$$\tilde{D} = \begin{pmatrix} \mathbf{1}_N^T & \mathbf{0}_M^T & 0 & \mathbf{0}_q^T & \mathbf{0}_{N+M}^T & \mathbf{0}_{N+M}^T \\ I_{N+M} & & \mathbf{0}_{N+M} & O_{(N+M)\times q} & I_{N+M} & -I_{N+M} \end{pmatrix}, \quad \tilde{E} = \begin{pmatrix} 1 \\ \mathbf{0}_{N+M} \end{pmatrix}.$$

Finally, the inequality matrix binds the unified shortfall distribution and enforces the leverage limit solely on the gross asset exposure:

$$\tilde{A} = \begin{pmatrix} -R^{\text{all}} & -\mathbf{1}_q & -I_q & O_{q\times(2N+2M)} \\ \mathbf{0}_{N+M}^T & 0 & \mathbf{0}_q^T & \begin{pmatrix} \mathbf{1}_N^T & \mathbf{0}_M^T \end{pmatrix} & \begin{pmatrix} \mathbf{1}_N^T & \mathbf{0}_M^T \end{pmatrix} \end{pmatrix}, \quad b = \begin{pmatrix} \mathbf{0}_q \\ \ell \end{pmatrix}.$$

Solving this comprehensive quadratic program yields the optimal regularized asset and currency allocations concurrently.